

2-Hour Consolidated Hackathon Plan (Participants)

Objective

Build a minimal document-validation app and prove quality with Playwright in a single 2-hour session.

Recommended scenario for this short format: build an HR form validator that accepts a submitted form and validates required fields and basic business rules.

By the end, you should have:

- A working form -> API -> validation response flow
- One passing Playwright happy-path test
- One passing Playwright negative-path test
- A simple 3-5 minute demo story

Timeboxed Agenda (120 Minutes)

Agenda context: this run is optimized for one clear use case, an HR form validation workflow where users submit form data and immediately receive validation results.

Example A (valid submission):

- Input: `employeeName="Maya Patel", employeeId="EMP-1042", department="Finance", startDate="2025-09-01", managerEmail="manager@contoso.com"`
- Expected result: ``valid: true`` and a success message such as "HR form is valid."

Example B (invalid submission):

- Input: `employeeName="", employeeId="1042", department="", startDate="2035-01-01", managerEmail="manager-at-contoso.com"`
- Expected result: ``valid: false`` with clear errors such as "Employee name is required", "Employee ID must match EMP-XXXX", "Department is required", "Start date cannot be in the future", and "Manager email format is invalid"

Block	Duration	Outcome
---	---	---
1) Kickoff + Scope Lock	10 min	Use case chosen, prompts drafted
2) Build Minimal App Flow	35 min	Frontend and API connected
3) Add Validation + Fixtures	15 min	Basic rules + reusable test data
4) Playwright Setup + Happy Path	30 min	1 stable happy-path test passing
5) Negative Path + Stabilize	20 min	1 negative-path test passing
6) Demo Prep + Final Run	10 min	Rehearsed demo and clean run

Block-by-Block Instructions

1) Kickoff + Scope Lock (0:00-0:10)

Pick one narrow scenario (invoice, HR form, ID check, or certificate).

Use Copilot quickly:

1. Ask Plan agent for architecture and step order.
2. Ask Agent mode to generate short execution prompts.
3. Save prompts in `phase-short-prompts.md`.

Scope rules:

- One form, one API endpoint, 2-3 validation rules.
- No database, authentication, or advanced UI.

2) Build Minimal App Flow (0:10-0:45)

Implement only what is needed for an end-to-end path:

- Frontend form with required fields
- Submit action to API (for example `/validate`)
- API returns JSON with `valid`, `message`, and optional `errors`
- Show result in UI

Checkpoint at 0:45:

- Submitting a valid input displays a success result.

3) Add Validation + Fixtures (0:45-1:00)

Add 2-3 rules such as:

- required fields present
- amount must be positive
- date is not in the future

Create test fixtures in `test-data/`:

- `test-data/valid/valid-1.json`
- `test-data/invalid/missing-field.json`
- `test-data/invalid/bad-value.json`

Checkpoint at 1:00:

- Invalid submissions show clear error text.

4) Playwright Setup + Happy Path (1:00-1:30)

Set up Playwright and write one happy-path test.

Test must cover:

- Load page
- Fill form with valid data
- Submit
- Assert clear success output

Use stable locators first:

- `getByRole`
- `getByLabel`
- `getByText`

Checkpoint at 1:30:

- Happy-path test passes consistently.

5) Negative Path + Stabilize (1:30-1:50)

Add one negative-path test (missing field or invalid value).

Assertions required:

- Expected validation error is visible
- Success message is not shown

Stabilize:

- Remove brittle selectors
- Tighten assertions to exact expected text where practical

Checkpoint at 1:50:

- Negative-path test passing.

6) Demo Prep + Final Run (1:50-2:00)

Stop adding features. Rehearse a 3-5 minute demo:

1. Show app purpose and one valid submission.
2. Run tests (happy + negative).
3. Show one Copilot moment that accelerated delivery.

Final verification:

- App starts cleanly.
- Tests run without manual tweaks.

Minimum Definition of Done

- Working app flow (form -> API -> visible result)
- At least 2 validation rules implemented
- 2 Playwright tests passing (1 happy, 1 negative)
- Fixtures organized under `test-data/`
- Demo story rehearsed

If You Fall Behind

Prioritize in this order:

1. Working app flow
2. One happy-path test passing
3. One negative-path test passing

Skip all stretch scope until those three are done.